



IF185301 TESIS - PROPOSAL

**PENGAMBILAN KEPUTUSAN DAN
PENGATURAN SKALABILITAS ELASTIS PADA
LINGKUNGAN CLOUD YANG HETEROGEN
DENGAN BERBASISKAN PADA PREDIKSI
WORKLOAD**

**MUHAMMAD ZAHID MASRURI
6025222041**

Dosen Pembimbing:

Prof. Tohari Ahmad, S.Kom., M.IT., Ph.D.

Royyana Muslim Ijtihadie, S.Kom., M.Kom., Ph.D.

**PROGRAM MAGISTER
BIDANG KEAHLIAN TEKNOLOGI JARINGAN DAN KEAMANAN SIBER
CERDAS (NETICS)
PROGRAM STUDI S2 TEKNIK INFORMATIKA
DEPARTEMEN TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI ELEKTRO DAN INFORMATIKA CERDAS
INSTITUT TEKNOLOGI SEPULUH NOPEMBER
SURABAYA
2026**



IF185301 TESIS - PROPOSAL

**PENGAMBILAN KEPUTUSAN DAN
PENGATURAN SKALABILITAS ELASTIS PADA
LINGKUNGAN CLOUD YANG HETEROGEN
DENGAN BERBASISKAN PADA PREDIKSI
WORKLOAD**

**MUHAMMAD ZAHID MASRURI
6025222041**

Dosen Pembimbing:

Prof. Tohari Ahmad, S.Kom., M.IT., Ph.D.

Royyana Muslim Ijtihadie, S.Kom., M.Kom., Ph.D.

**PROGRAM MAGISTER
BIDANG KEAHLIAN TEKNOLOGI JARINGAN DAN KEAMANAN SIBER
CERDAS (NETICS)
PROGRAM STUDI S2 TEKNIK INFORMATIKA
DEPARTEMEN TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI ELEKTRO DAN INFORMATIKA CERDAS
INSTITUT TEKNOLOGI SEPULUH NOPEMBER
SURABAYA
2026**

LEMBAR PENGESAHAN

PROPOSAL TESIS

Judul : Pengambilan Keputusan dan Pengaturan Skalabilitas Elastis pada Lingkungan Cloud yang Heterogen dengan Berbasis pada Prediksi Workload

Mahasiswa : Muhammad Zahid Masruri

NRP : 6025222041

Telah diseminarkan pada,

Hari : Rabu

Tanggal : 10 Juli 2024

Tempat : Ruang 217B

Mengetahui/Menyetujui,

Dosen Penguji:

1.

Dosen Penguji ke-1 (Lengkap)

NIP: 20xxxxxxxxx

2.

Dosen Penguji ke-2 (Lengkap)

NIP: 20xxxxxxxxx

3.

Dosen Penguji ke-3 (Lengkap)

NIP: 20xxxxxxxxx

Dosen Pembimbing:

1.

Prof. Tohari Ahmad, S.Kom., M.IT., Ph.D.

NIP: 197708242006041001

2.

Royyana Muslim Ijtihadie, S.Kom., M.Kom.,
Ph.D.

NIP: 197505252003121002

Halaman ini sengaja dikosongkan.

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah SWT yang telah memberikan rahmat serta kemudahan sehingga peneliti dapat menyelesaikan penelitian tesis dengan judul “Pengambilan Keputusan dan Pengaturan Skalabilitas Elastis pada Lingkungan Cloud yang Heterogen dengan Berbasis pada Prediksi Workload” ini dapat terselesaikan.

Peneliti menyadari bahwa tesis ini tidak akan berhasil tanpa adanya bantuan dari beberapa pihak. Oleh karena itu, penulis ingin mengucapkan terima kasih kepada:

1. Bapak Prof. Tohari Ahmad, S.Kom., M.IT., Ph.D. selaku dosen pembimbing tesis 1 yang telah sabar dalam membimbing dan mengarahkan peneliti sehingga dapat menyelesaikan tesis ini.
2. Bapak Royyana Muslim Ijtihadie, S.Kom., M.Kom., Ph.D. selaku dosen pembimbing tesis 2 yang telah sabar dalam membimbing dan mengarahkan peneliti sehingga dapat menyelesaikan tesis ini.
3. Orang tua dari peneliti yang telah memberikan dukungan, nasihat, kasih sayang, dan perhatian dalam mendidik dan membantu penulis, sehingga dapat menyelesaikan tesis ini.
4. Semua pihak lainnya yang secara langsung maupun tidak langsung dalam membantu peneliti menyelesaikan tesis ini.

Peneliti menyadari bahwa dalam penyusunan laporan tesis ini masih terdapat banyak kekurangan, sehingga kritik dan saran sangat diharapkan. Akhir kata peneliti berharap tesis ini dapat membawa manfaat bagi semua pihak yang menggunakannya.

Surabaya, 13 Mei 2024

Muhammad Zahid Masruri
NRP: 6025222041

Halaman ini sengaja dikosongkan.

ABSTRAK

Pengambilan Keputusan dan Pengaturan Skalabilitas Elastis pada Lingkungan Cloud yang Heterogen dengan Berbasis pada Prediksi Workload

Muhammad Zahid Masruri / NRP 6025222041

Arsitektur cloud modern telah mengubah cara aplikasi web dikembangkan, digunakan, dan dikelola, menawarkan fleksibilitas, skalabilitas, dan efisiensi yang signifikan. Evolusi ini mencakup pergeseran dari infrastruktur tradisional seperti mesin virtual ke teknologi kontainer dan model serverless. Kubernetes, sebagai sistem manajemen kluster berbasis kontainer, populer karena kemampuannya untuk skala berdasarkan permintaan. Namun, algoritma skalabilitas tradisional sering gagal menangani lonjakan lalu lintas mendadak, yang menyebabkan peningkatan latensi dan alokasi sumber daya yang tidak efisien. Sebagai solusi, komputasi serverless atau Function as a Service (FaaS) menyediakan skalabilitas otomatis dan efisiensi biaya dengan mengenakan biaya hanya berdasarkan penggunaan aktual, meskipun model ini menimbulkan biaya tinggi untuk beban kerja konstan.

Penelitian ini mengeksplorasi integrasi antara Kubernetes dan komputasi serverless untuk mengoptimalkan kinerja dan efisiensi biaya aplikasi cloud. Meskipun beberapa penelitian telah fokus pada prediksi alokasi sumber daya, masih terdapat tantangan dalam pengelolaan lonjakan lalu lintas dan alokasi sumber daya yang belum sepenuhnya teratasi. Penelitian ini mengusulkan strategi distribusi lalu lintas yang memanfaatkan prediksi beban kerja berbasis GRU dan mekanisme reservasi sumber daya untuk memastikan alokasi yang optimal dan pemenuhan tail latency. Dengan menggabungkan keunggulan dari Kubernetes dan serverless, penelitian ini bertujuan untuk menciptakan arsitektur cloud yang lebih tangguh, responsif, dan efisien biaya.

Kata Kunci: Cloud Computing, Gated Recurrent Unit, Serverless, Function

as A Service, Container, Kubernetes

Halaman ini sengaja dikosongkan.

ABSTRACT

Decision Making and Elastic Scalability Management in Heterogeneous Cloud Environments Based on Workload Prediction

Muhammad Zahid Masruri / NRP 6025222041

Modern cloud architecture has transformed the way web applications are developed, used, and managed, offering significant flexibility, scalability, and efficiency. This evolution includes a shift from traditional infrastructure such as virtual machines to container technology and serverless models. Kubernetes, a container-based cluster management system, is popular for its ability to scale on demand. However, traditional scalability algorithms often fail to handle sudden traffic spikes, resulting in increased latency and inefficient resource allocation. As a solution, serverless computing or Function as a Service (FaaS) provides automatic scalability and cost efficiency by charging only based on actual usage, though this model incurs high costs for constant workloads.

This research explores the integration between Kubernetes and serverless computing to optimize the performance and cost efficiency of cloud applications. While several studies have focused on resource allocation prediction, challenges in managing traffic spikes and dynamic resource allocation remain unresolved. This study proposes a traffic distribution strategy utilizing GRU-based workload prediction and a resource reservation mechanism to ensure optimal allocation and fulfillment of tail latency. By combining the strengths of Kubernetes and serverless, this research aims to create a more resilient, responsive, and cost-efficient cloud architecture.

Keywords: Cloud Computing, Gated Recurrent Unit, Serverless, Function as A Service, Container, Kubernetes

Halaman ini sengaja dikosongkan.

DAFTAR ISI

LEMBAR PENGESAHAN PROPOSAL TESIS	i
KATA PENGANTAR	iii
ABSTRAK	v
ABSTRACT	viii
DAFTAR ISI	x
DAFTAR TABEL	xiii
DAFTAR GAMBAR	xv
1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	4
1.3 Tujuan Penelitian	4
1.4 Manfaat Penelitian	5
1.5 Kontribusi Penelitian	5
1.6 Batasan Masalah	5
2 KAJIAN PUSTAKA	7
2.1 Cloud Computing	7
2.1.1 Container & Kubernetes	9
2.1.2 Function as a Service and Serverless	10
2.2 Integrasi Antara Kluster Kubernetes dan Serverless	11

2.3	Cloud Application Performance	12
2.3.1	SLA Violation	12
2.3.2	Request Response Time	12
2.3.3	Resource Utilization	13
2.4	Algorithm and Prediction Model	13
2.4.1	LSTM	13
2.4.2	GRU	13
2.5	Kubernetes Scalling Strategy	14
2.5.1	Kluster & Pod Autoscaler	14
2.5.2	Metode ElaX dalam Scalling Kluster	14
3	METODOLOGI	17
3.1	Studi Literatur	18
3.2	Pengumpulan Data	18
3.3	Perancangan Metode	19
3.4	Implementasi Metode	19
3.4.1	Load Predictor	20
3.4.2	Resource Alocator	20
3.4.2.1	Tuning Koefisien	21
3.4.3	Online Controller	21
3.5	Rencana Evaluasi	21
3.5.1	Evaluasi Traffic Predictor	23
3.5.2	Evaluasi Router dan Scalling	23
3.6	Penyusunan Laporan	24
3.7	Rencana dan Jadwal Kegiatan Penelitian	24
	DAFTAR PUSTAKA	25

Halaman ini sengaja dikosongkan.

DAFTAR TABEL

3.1	Contoh dataset request	19
3.2	Rencana Kegiatan Penelitian Thesis	24

Halaman ini sengaja dikosongkan.

DAFTAR GAMBAR

2.1	Cloud services and division of responsibilities (Kavis, 2014) . .	8
2.2	Architecture of a Kubernetes cluster	9
2.3	Serverless Architecture (Mampage et al., 2022)	11
2.4	Arsitektur ElaX	15
2.5	Online Control Algorithm	16
3.1	Alur penelitian tesis	17
3.2	Routing Controller	22

Halaman ini sengaja dikosongkan.

BAB 1

PENDAHULUAN

Bab ini membahas mengenai hal-hal yang menjadi dasar dari penelitian. Hal tersebut meliputi latar belakang yang membahas mengenai identifikasi masalah penelitian beserta referensi penelitian sebagai penelitian terdahulu, rumusan masalah, tujuan penelitian, batasan masalah, hingga manfaat penelitian.

1.1 Latar Belakang

Arsitektur cloud modern mengubah bagaimana aplikasi web dikembangkan, digunakan, dan dikelola; menawarkan tingkat fleksibilitas, skalabilitas, dan efisiensi yang belum pernah terjadi sebelumnya (Smit, 2013). Evolusi ini ditandai dengan pergeseran dari penyediaan infrastruktur dasar seperti mesin virtual (Virtual Machine atau VM) atau Virtual Private Server (VPS), menjadi lebih skalabel dengan Container, dan edge worker yang tersebar menggunakan model serverless. Ini didorong oleh kebutuhan akan sumber daya komputasi yang lebih efisien, pengurangan biaya operasional, dan peningkatan kinerja serta skalabilitas untuk mengakomodasi kompleksitas dan basis pengguna yang berkembang karena model tradisional kesulitan untuk mengikuti.

Kubernetes adalah sistem manajemen cluster berdasarkan teknologi container. Popularitasnya merupakan hasil dari kemampuan skalabilitas berdasarkan permintaan. Namun, algoritma skalabilitas tradisional yang ditawarkan oleh penyedia layanan cloud tidak dapat memenuhi lonjakan lalu lintas mendadak karena waktu startup container yang lambat, sering kali menyebabkan pen-

ingkatan latensi dan alokasi sumber daya berlebihan yang tidak terpakai ketika lalu lintas menurun (Yang et al., 2019). Penundaan waktu respons aplikasi dapat merusak pengalaman pengguna, akibatnya menyebabkan kerugian dalam bisnis.

Bersamaan dengan Kubernetes, komputasi Serverless, atau Function as a Service (FaaS) telah mendapatkan traksi dalam beberapa tahun terakhir. Model ini menangani penyediaan dan scaling infrastruktur secara otomatis, memungkinkan aplikasi dapat berjalan dengan tingkat ketersediaan tinggi tanpa kerumitan dalam pengelolaan server. Sehingga pengembang dan organisasi dapat sepenuhnya fokus pada layanan aplikasi (Sadaqat et al., 2018; Savage, 2018; Sewak and Singh, 2018). Selain itu, berdasarkan benchmark Yu et al. (2020) fungsi serverless dapat diskalakan dari nol ke kapasitas target dalam hitungan detik, dengan rata-rata waktu cold start dari 300 ms hingga 8 detik untuk sebagian besar platform tergantung pada runtime yang digunakan.

Dalam konteks evolusi arsitektur cloud yang terus berkembang, muncul tantangan signifikan yang melibatkan tradeoff antara Skalabilitas dan Efisiensi, di mana upaya mencapai skalabilitas tinggi untuk menanggulangi lonjakan lalu lintas yang tak terprediksi sering kali bertentangan dengan efisiensi penggunaan sumber daya untuk mengoptimalkan biaya. Strategi skalabilitas tradisional, seperti over-provisioning kluster Kubernetes, dengan cara mengalokasikan sumber daya berlebih pada sebuah kluster, untuk menangani lonjakan lalu lintas yang tidak terprediksi, mengakibatkan penggunaan sumber daya yang kurang optimal ketika tidak terdapat lalu lintas tinggi, meningkatkan biaya tanpa meningkatkan performa (Zhang et al., 2021).

Untuk mengevaluasi dan mengoptimalkan aplikasi cloud secara efektif, beberapa metrik digunakan. Tail latency, mengukur waktu yang diperlukan untuk memproses permintaan yang paling lambat dalam sistem, digunakan untuk memahami kinerja terburuk dan memastikan bahwa semua permintaan pengguna memenuhi waktu respons yang dapat diterima (Aslanpour et al., 2020). Running time, durasi total untuk menjalankan tugas atau fungsi ter-

tentu. Alokasi sumber daya yang efektif, termasuk distribusi CPU, memori, dan penyimpanan, digabungkan dengan penggunaan CPU di seluruh cluster membantu mengidentifikasi kemungkinan ketidakefisienan sumber daya.

Perkembangan terbaru dalam komputasi awan telah memperkenalkan berbagai metode untuk mengatasi masalah latensi dan skalabilitas pada arsitektur Kubernetes dan Serverless. Yang et al. (2019) memanfaatkan algoritma pembelajaran mesin berbasis LSTM (Long Short-Term Memory) untuk melakukan peramalan beban kerja sebagai bagian dari algoritma yang diusulkan. Hasilnya menunjukkan kurang dari 18% sumber daya yang mengalami over-provisioning. Yuan and Liao (2024) memanfaatkan peramalan deret waktu untuk mengantisipasi kebutuhan sumber daya, secara signifikan memangkas waktu cold start serverless. Sementara itu, He (2020) menggunakan algoritma prediktif untuk mengoptimalkan jumlah instansi sebelum terjadi fluktuasi permintaan. Mondal et al. (2023) memanfaatkan model GRU (Gated Recurrent Unit) untuk memprediksi beban kerja Kubernetes secara custom.

LSTM (Long Short-Term Memory) adalah jenis jaringan saraf dalam yang dirancang khusus untuk mengatasi masalah pada data deret waktu. LSTM sangat berguna untuk kasus-kasus prediksi di mana data memiliki dependensi temporal yang panjang, seperti dalam prediksi beban kerja server cloud. Selain LSTM, terdapat juga Gated Recurrent Unit (GRU) yang memiliki struktur lebih sederhana dan memerlukan sumber daya lebih sedikit dibandingkan LSTM, namun tetap mampu memberikan performa prediksi yang tidak identik (Mondal et al., 2023). GRU memilih informasi yang relevan untuk disimpan atau dibuang, menghasilkan prediksi yang lebih efisien dalam konteks komputasi awan.

Untuk membuat model prediksi dalam cloud computing yang efektif, diperlukan dataset yang menggambarkan kondisi sistem. Yang et al. (2019) menggunakan dataset berupa volume request dari Calgary Trace dan ClarkNet dalam mengembangkan bagian traffic predictor pada metode scalling yang diusulkan. Calgary Trace menyediakan data lalu lintas jaringan dari Universi-

ty of Calgary, berisi lebih dari 4 juta request HTTP. ClarkNet dataset berfokus pada penggunaan web, menyediakan data rinci tentang permintaan HTTP ke server WWW ClarkNet.

Penelitian ini mengusulkan strategi distribusi lalu lintas antara kluster Kubernetes dan serverless, bertujuan untuk meningkatkan efisiensi biaya dan mengurangi waktu respons selama lonjakan lalu lintas jaringan yang tidak terduga. Berdasarkan metode Elax, studi ini akan menggabungkan Prediktor Beban Kerja berbasis GRU untuk peramalan beban kerja yang tepat, ditambah dengan mekanisme Reservasi Sumber Daya untuk alokasi sumber daya yang optimal. Sebuah Pengontrol akan melepaskan sumber daya yang tidak terpakai berdasarkan umpan balik dengan tetap memastikan persyaratan tail-latency terpenuhi.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dipaparkan, terdapat beberapa rumusan masalah yang dirumuskan sebagai berikut:

1. Bagaimana merancang prediksi beban traffic pada sebuah aplikasi dengan menggunakan GRU?
2. Bagaimana merancang dan melakukan pengambilan keputusan dengan melakukan modifikasi Elax untuk scaling pada server cluster dan mendistribusikan traffic ke jenis kluster yang berbeda?
3. Bagaimana mengevaluasi mekanisme Elax yang dimodifikasi untuk automatic scaling dan load distribution pada integrasi kubernetes dan serverless pada sebuah aplikasi?

1.3 Tujuan Penelitian

Berdasarkan rumusan masalah yang telah dijelaskan, tujuan besar dari penelitian tugas akhir ini adalah sebagai berikut:

1. Menganalisis dampak integrasi Kubernetes dan serverless dalam men-

gatasi tantangan efisiensi dan scalability aplikasi.

2. Menilai performa integrasi Kubernetes dan Serverless dalam meningkatkan kemampuan Scalling aplikasi sehingga bisa mengurangi masalah Latency dan Over-provisioning.
3. Mengukur dan mengevaluasi pendekatan ini dibandingkan dengan model lain dalam menangani permasalahan latency dan over-provisioning.
4. Mengembangkan metode scalling dan routing traffic yang mengintegrasikan Kubernetes dan Serverless.

1.4 Manfaat Penelitian

Adapun manfaat yang bisa diperoleh dari penelitian ini dipaparkan sebagai berikut:

1. Memahami bagaimana dynamic scaling and routing dengan menggabungkan Kubernetes dan Serverless dapat meningkatkan kemampuan Scalling dan efisiensi pada aplikasi.
2. Memberikan pemahaman dan wawasan tentang bagaimana dynamic scaling and routing dengan mengintegrasikan Kubernetes dan Serverless bisa mengatasi tantangan scalling dan efisiensi pada aplikasi.

1.5 Kontribusi Penelitian

Adapun kontribusi yang diperoleh dari penelitian ini sebagai berikut:

1. Merancang metode prediksi beban traffic pada aplikasi dengan mempertimbangkan history volume permintaan per detik.
2. Membuat sebuah metode scalling dan dynamic routing dengan menggabungkan Kluster Kubernetes dan Serverless yang berbasis metode Elax.

1.6 Batasan Masalah

Agar penelitian yang dilakukan dapat berjalan sesuai dengan tujuan yang dipaparkan, ditetapkan beberapa batasan masalah sebagai berikut:

1. Dataset yang digunakan merupakan dataset traffic dari ClarkNet dan Calgary trace.
2. Cloud provider yang digunakan adalah Google Cloud dengan menggunakan Google Kubernetes Engine untuk Kubernetes dan Cloud Function untuk Serverless deployment.
3. Basis metode dan algoritma yang digunakan ada Elax dengan modifikasi pada output yang menambahkan decision untuk routing ke Serverless, Pre-Emptive Cold Start, dan Tuning Scaling logic.

BAB 2

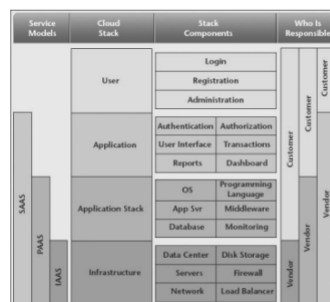
KAJIAN PUSTAKA

Pada bab ini dijelaskan mengenai pustaka yang digunakan sebagai landasan ilmiah penelitian dan pustaka penelitian yang telah dilakukan sebelumnya terkait penelitian yang akan dilakukan.

2.1 Cloud Computing

Cloud computing, menurut National Institute of Standards and Technology (NIST), adalah model yang menyediakan akses on-demand ke kumpulan jaringan komputasi yang dapat dikonfigurasi. Model ini mencakup tiga layanan utama: Infrastructure-as-a-Service (IaaS) yang menawarkan sumber daya virtual yang dapat diskalakan; Platform-as-a-Service (PaaS) yang menyediakan alat dan middleware untuk pengembang; dan Software-as-a-Service (SaaS) yang menyediakan aplikasi perangkat lunak melalui internet (Mell and Grance, 2011). Peralihan dari komputasi tradisional ke model berbasis layanan ini didorong oleh kemajuan dalam virtualisasi, penyimpanan, dan daya pemrosesan (Malathi, 2011).

Teknologi kontainer, yang didukung oleh platform seperti Docker, memudahkan pengelolaan kontainer dan membuka jalan bagi alat orkestrasi seperti Kubernetes (Yadav et al., 2019). Kubernetes telah merevolusi orkestrasi kontainer dengan mengotomatiskan penerapan, penskalaan, dan manajemen aplikasi yang terkontainerisasi (Pahl et al., 2019). Selain itu, model komputasi tanpa server seperti Function as a Service (FaaS) lebih lanjut menyederhanakan manajemen infrastruktur, memungkinkan pengembang fokus pada

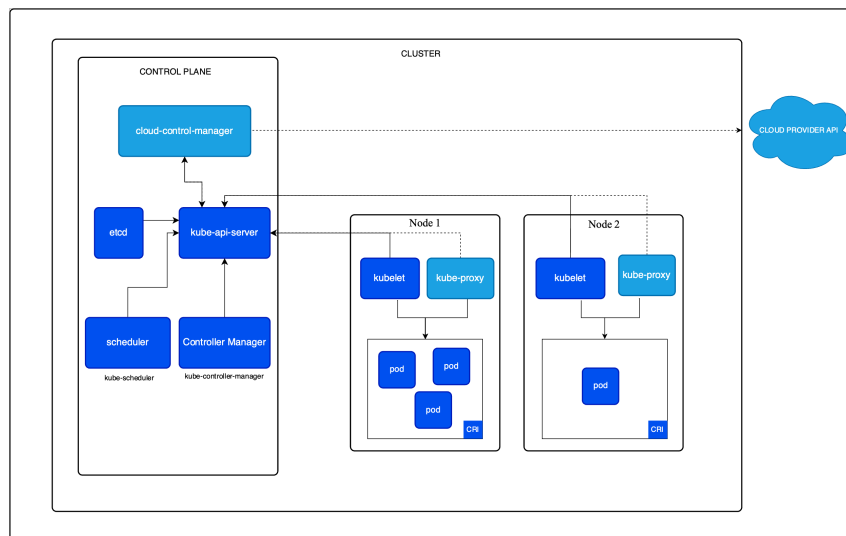


Gambar 2.1 Cloud services and division of responsibilities (Kavis, 2014)

penulisan kode (Wurster et al., 2018).

2.1.1 Container & Kubernetes

Kontainer telah menjadi teknologi utama dalam komputasi awan, menawarkan alternatif yang lebih efisien dibandingkan mesin virtual tradisional (VM). Berbeda dengan VM yang memerlukan sistem operasi lengkap, kontainer berbagi kernel dengan sistem host dan mengisolasi lingkungan eksekusi aplikasi, mengurangi overhead dan meningkatkan kinerja. Isolasi ini dicapai melalui namespace dan control group—fitur Linux yang dapat membatasi akses dan penggunaan sebuah proses, memungkinkan multi-tenancy yang aman dan kontrol alokasi sumber daya. Docker, menyediakan platform yang menyederhanakan manajemen package, distribusi, dan manajemen aplikasi yang terkontainerisasi.



Gambar 2.2 Architecture of a Kubernetes cluster

Seiring dengan meluasnya penggunaan kontainer, kebutuhan akan orkestrasi mereka memunculkan Kubernetes. Kubernetes adalah platform open-source yang dirancang untuk mengotomatisasi penerapan, penskalaan, dan operasi kontainer aplikasi. Platform ini mengelompokkan kontainer yang membentuk aplikasi ke dalam unit logis untuk memudahkan manajemen dan penemuan. Kubernetes menyediakan kerangka kerja untuk menjalankan sistem terdis-

tribusi secara andal, dengan penemuan layanan dan penyeimbangan beban, orkestrasi penyimpanan, pengguliran otomatis dan pembatalan pengguliran, serta kemampuan penyembuhan diri.

Komponen utama dari control plane meliputi:

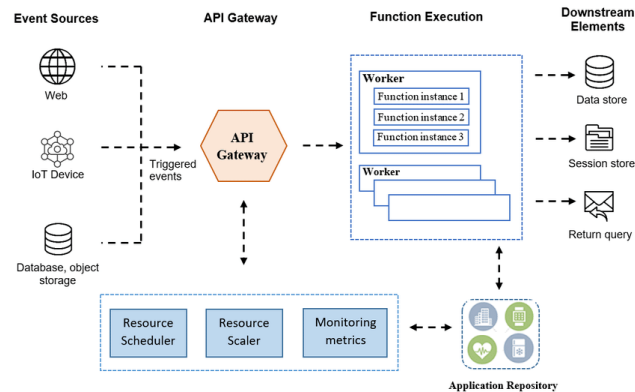
- etcd: Penyimpanan key-value yang persisten, ringan, dan terdistribusi, menyimpan data konfigurasi, status, dan metadata kluster.
- kube-api-server: Server API yang berfungsi sebagai antarmuka depan untuk control plane.
- kube-scheduler: Komponen yang memantau pod baru yang belum memiliki node dan memilih node berdasarkan kriteria penjadwalan.
- kube-controller-manager: Menjalankan proses kontrol yang mengatur status kluster.
- cloud-control-manager: Menghubungkan kluster dengan API penyedia cloud.

Pada tingkat node, komponen utama meliputi:

- kubelet: Agen yang berjalan di setiap node yang memastikan kontainer berjalan dalam pod.
- kube-proxy: Proxy jaringan yang menjaga aturan jaringan untuk komunikasi pod.
- Container Runtime Interface (CRI): Antarmuka untuk runtime kontainer agar terintegrasi dengan kubelet.
- Pod: Unit terkecil yang dapat dibuat dan dikelola di Kubernetes.

2.1.2 Function as a Service and Serverless

Function as a Service (FaaS) atau serverless, merupakan model layanan cloud yang memungkinkan pengembang untuk menerapkan fungsi individu—blok kode yang dipicu oleh peristiwa tertentu. FaaS menyembunyikan infrastruktur dasar, termasuk server, sistem operasi, dan bahkan lingkungan eksekusi aplikasi, sehingga pengembang dapat fokus hanya pada penulisan logika bisnis.



Gambar 2.3 Serverless Architecture (Mampage et al., 2022)

Dalam arsitektur serverless, penyedia cloud bertanggung jawab untuk menjalankan fungsi sebagai respons terhadap peristiwa dan sepenuhnya mengelola infrastruktur yang mendasarinya. Pelanggan dikenakan biaya berdasarkan jumlah sumber daya yang benar-benar digunakan oleh aplikasi mereka, bukan berdasarkan unit kapasitas yang telah dibeli sebelumnya. Model ini dapat menghasilkan penghematan biaya yang signifikan karena menghilangkan kebutuhan untuk menjalankan kapasitas server yang tidak digunakan secara terus-menerus, namun akan menjadi mahal apabila terdapat beban konstan yang memerlukan sumber daya secara terus menerus.

2.2 Integrasi Antara Kluster Kubernetes dan Serverless

Seiring dengan pertumbuhan komputasi serverless, interaksi antara Function as a Service (FaaS) dan Kubernetes menjadi semakin relevan. Sementara Kubernetes mengorkestrasi kontainer dengan efisien, komputasi serverless dapat menangani lonjakan peristiwa dengan cepat tanpa perlu mengelola server secara langsung. Mengintegrasikan keunggulan Kubernetes dan FaaS memungkinkan mekanisme distribusi lalu lintas yang secara dinamis dapat menyesuaikan skala dengan permintaan beban kerja, sambil mengoptimalkan biaya dan kinerja.

Strategi integrasi melibatkan konfigurasi deployment di mana Kubernetes mengelola layanan jangka panjang yang konstan dan predictable, sementara

fungsi serverless menangani tugas-tugas singkat ketika layanan kubernetes belum melakukan scalling sehingga tidak dapat dilayani. Model hibrida ini berpotensi menawarkan yang terbaik dari kedua dunia: kapabilitas orkestrasi dan manajemen Kubernetes serta model on-demand dan pay-as-you-go dari serverless.

2.3 Cloud Application Performance

Menentukan seberapa layanan cloud memenuhi ekspektasi pengguna dan Service Level Agreements (SLA). Untuk memastikan bahwa aplikasi cloud beroperasi secara optimal, beberapa metrik kunci digunakan untuk mengevaluasi berbagai aspek kinerja.

2.3.1 SLA Violation

Pelanggaran SLA adalah salah satu indikator utama dalam penilaian kinerja aplikasi cloud. Pelanggaran ini terjadi ketika layanan gagal memenuhi ketentuan yang disepakati dalam SLA. Untuk mengukur pelanggaran SLA, dua metrik utama yang digunakan adalah tingkat keberhasilan dan jumlah tugas yang gagal (Aslanpour et al., 2020).

2.3.2 Request Response Time

Waktu respons permintaan mencerminkan seberapa cepat aplikasi cloud dapat merespons permintaan pengguna. Metrik ini mencakup beberapa aspek, yaitu waktu berjalan, keterlambatan, dan tail latency. Waktu berjalan mengukur durasi total untuk menyelesaikan suatu tugas. Keterlambatan diukur sebagai selisih antara waktu penyelesaian aktual dan yang diharapkan. Tail latency, yang merupakan waktu respons di persentil tertinggi, memberikan gambaran tentang performa dalam kondisi beban puncak.

2.3.3 Resource Utilization

Resource Utilization mencakup alokasi, utilisasi, dan biaya. Alokasi sumber daya mengacu pada penugasan CPU, memori, dan bandwidth yang optimal. Utilisasi sumber daya diukur sebagai persentase dari kapasitas total yang digunakan. Biaya terkait dengan penggunaan sumber daya dihitung berdasarkan model pay-as-you-go.

2.4 Algorithm and Prediction Model

Dalam konteks manajemen lalu lintas dan penskalaan dinamis pada layanan cloud containerized, algoritma dan model prediksi memainkan peran penting dalam memprediksi beban kerja dan mengoptimalkan alokasi sumber daya.

2.4.1 LSTM

LSTM adalah pengembangan dari recurrent neural network (RNN) yang dirancang untuk dapat belajar dan mengingat hubungan serta pola antara data yang berurutan dalam jangka panjang. Design dari LSTM memungkinkan untuk mengatasi permasalahan vanishing gradient pada jaringan RNN. Satu sel LSTM terdiri dari gerbang input, output, dan forget yang berfungsi mengontrol aliran data.

2.4.2 GRU

Gated Recurrent Unit (GRU) adalah varian dari RNN yang lebih sederhana dibandingkan LSTM, dengan mekanisme kontrol yang lebih sedikit. Terdiri dari hanya dua gerbang, sehingga lebih cepat dalam proses training dan inference, dengan tetap mempertahankan kemampuan untuk menangani ketergantungan jangka panjang (Mondal et al., 2023). Update gate menggabungkan fungsi dari input dan forget gate pada LSTM, bertanggung jawab mengatur informasi baru dan yang akan disimpan dalam sel. Reset gate mengatur

bagaimana informasi baru diintegrasikan dengan data sebelumnya.

2.5 Kubernetes Scalling Strategy

Strategi scaling pada aplikasi berfungsi agar aplikasi dapat menangani beban kerja yang berubah-ubah secara efisien. Kubernetes mendukung strategi scalling secara horizontal, yang menambah jumlah deployment, dan scalling secara vertikal, yang menyesuaikan alokasi sumber daya dari pod yang ada.

2.5.1 Kluster & Pod Autoscaler

1. Cluster Autoscaler

Cluster Autoscaler adalah mekanisme yang menyesuaikan jumlah node dalam kluster Kubernetes berdasarkan kebutuhan sumber daya pod. Jika pod baru tidak dapat dijadwalkan karena kekurangan sumber daya, Cluster Autoscaler akan menambahkan node baru. Sistem ini memerlukan waktu 2-5 menit dalam melakukan perubahan terhadap alokasi sumber daya.

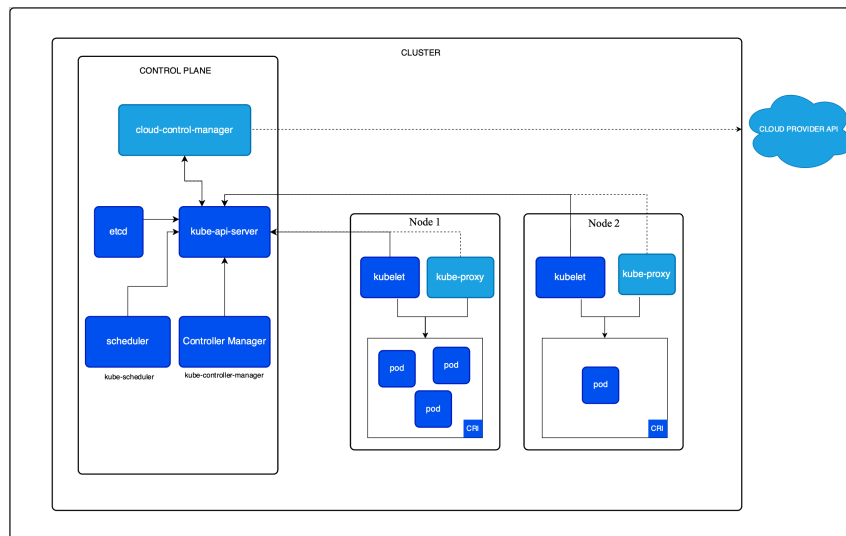
2. Pod Autoscaler

Horizontal Pod Autoscaler (HPA) secara otomatis akan menambah atau mengurangi jumlah replika pod berdasarkan metrik seperti CPU dan penggunaan memori. Scalling cenderung lebih responsif terhadap perubahan beban kerja, dapat berubah kurang dari 30 detik.

Vertical Pod Autoscaler (VPA) menyesuaikan alokasi sumber daya (CPU dan memori) dari pod yang ada berdasarkan kebutuhan beban kerja.

2.5.2 Metode ElaX dalam Scalling Kluster

ElaX adalah kerangka kerja penskalaan yang bertujuan untuk mengoptimalkan penyediaan sumber daya bagi layanan cloud containerized, dengan tetap menjamin pemenuhan Service Level Objectives (SLO) khususnya terkait dengan tail latency. Dalam penelitian ini ElaX digunakan sebagai basis metode yang akan dimodifikasi. Terdiri dari 3 komponen utama, ditunjukkan pada



Gambar 2.4 Arsitektur ElaX

Gambar 2.4.

1. Prediktor Beban Kerja

Tersusun dari jaringan Long Short-Term Memory (LSTM) untuk memprediksi beban kerja. Prediktor akan belajar dari data historis beban dan membuat prediksi pada beban kerja kedepan. Output prediksi beban kerja dari bagian ini akan masuk ke komponen reservasi sumber daya.

2. Reservasi Sumber Daya

Merupakan model workload-resource yang berfungsi untuk mengestimasi kebutuhan sumberdaya berdasarkan hasil prediksi beban kerja. Terdiri dari model pemrograman matematis yang mempertimbangkan cost ketika melakukan operasi scalling. Kemudian model akan mengeluarkan keputusan untuk melakukan scale-up dan scale-out dengan cost paling sedikit.

3. Pengendali Online

Kontroler bekerja berdasarkan nilai kondisi sekarang dibandingkan dengan SLO. Ketika tail latency berada pada level kritis akan melanggar SLO, maka sumber daya akan disesuaikan, sebaliknya apabila telah berada pada level yang aman, maka reservasi sumber daya akan dilepas. Algoritma 2.5 menunjukkan mekanisme pengendali online.

```

1: Variables:
2:    $SLO\_target \leftarrow \text{False}$ 
3:    $slack \leftarrow 0$ 
4:    $cur\_resource$ 
5: repeat
6:    $slack \leftarrow (SLO\_target - \text{GetTailLatency}()) / SLO\_target$ 
7:   if  $slack > 0$  then
8:      $\text{IncrResource}(cur\_resource \times 10\%)$ 
9:   else if  $0 < slack < 0.05$  then
10:     $\text{IncrResource}(cur\_resource \times 5\%)$ 
11:   else
12:     $extra\_resource \leftarrow cur\_resource - pre\_resource$ 
13:    if  $extra\_resource > 0$  then
14:       $\text{RemoveResource}(extra\_resource \times 50\%)$ 
15:    end if
16:   end if
17:   wait(2) ▷ Delay for 2 seconds
18: until forever

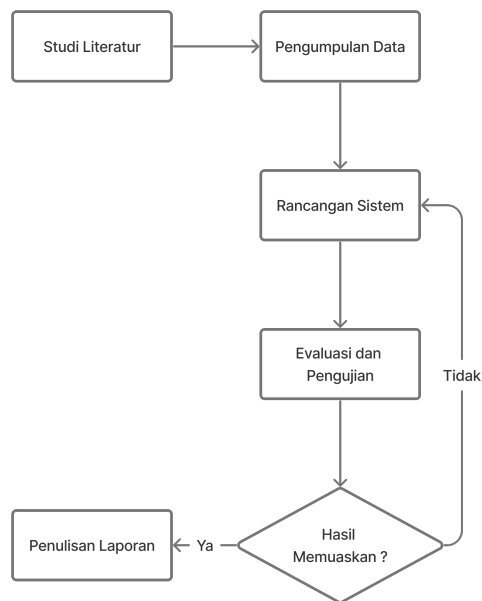
```

Gambar 2.5 Online Control Algorithm

BAB 3

METODOLOGI

Bab ini menjelaskan mengenai metodologi yang digunakan untuk menjawab rumusan masalah beserta tujuan penelitian yang dipaparkan sebelumnya. Adapun secara garis besar metodologi penelitian yang digunakan peneliti diilustrasikan seperti pada Gambar 3.1.



Gambar 3.1 Alur penelitian tesis

Penelitian ini terdiri dari beberapa tahapan yang terdiri dari 1) studi literatur, 2) pengumpulan data, 3) rancangan sistem, 4) pengujian, 5) analisis hasil, dan 6) penulisan laporan. Setiap tahapan metodologi penelitian ini akan dijelaskan dengan lebih rinci pada setiap subbab.

3.1 Studi Literatur

Studi literatur merupakan tahapan awal dalam penelitian ini. Studi literatur dilakukan untuk mencari referensi dari berbagai sumber mengenai penelitian yang terkait dan memiliki kemiripan dengan penelitian yang dilakukan. Studi literatur juga perlu dilakukan agar bisa memahami metode dan konsep sesuai pembahasan dan permasalahan yang akan diselesaikan sehingga bisa memberi solusi. Adapun literatur utama terkait dengan metode scaling pada Kubernetes dan Serverless mengacu pada penelitian yang dilakukan oleh Senjab et al. (2023), dan Mampage et al. (2022). Sedangkan untuk algoritma Elax yang akan dijadikan dasar metode untuk dimodifikasi mengacu pada penelitian Yang et al. (2019). Salah satu modifikasi yang dilakukan adalah mengganti bagian load predictor dari LSTM menjadi GRU.

3.2 Pengumpulan Data

Pada tahapan ini, dilakukan pengumpulan data untuk training prediktor beban traffic dan simulasi load testing pada aplikasi. Data training digunakan untuk melatih prediktor time-series berbasis GRU. Adapun data yang akan digunakan pada penelitian ini adalah data traffic load pada ClarkNet dan Calgary trace. Struktur data terdiri dari:

1. host yang meminta request. Dapat berupa nama host, atau IP address.
2. Timestamp dalam format “DAY MON DD HH:MM:SS YYYY”, dimana:
 - (a) DAY: hari dalam pekan
 - (b) MON: nama bulan
 - (c) DD: tanggal dalam bulan
 - (d) HH:MM:SS: waktu pada hari dengan format 24 jam
 - (e) YYYY: tahun
3. request metode dan URL request untuk HTTP protocol
4. HTTP reply code
5. Bytes: Ukuran bytes dalam reply

Data merupakan log seluruh request HTTP ke ClarkNet WWW dan University of Calgary's Department of Computer Science, dengan total data 4.055.326 requests. Berikut contoh sampel data request yang digunakan pada Tabel 3.1.

Tabel 3.1 Contoh dataset request

Dataset	Request
University of Calgary's Department of Computer Science	local - - [24/Oct/1994:13:41:41 -0600] "GET index.html HTTP/1.0" 200 150
	local - - [24/Oct/1994:13:41:41 -0600] "GET 1.gif HTTP/1.0" 200 1210
	local - - [24/Oct/1994:13:43:13 -0600] "GET index.html HTTP/1.0" 200 3185
ClarkNet	204.249.225.59 - - [28/Aug/1995:00:00:34 -0400] "GET /pub/rmharris/catalogs/dawsocat/intro.html HTTP/1.0" 200 3542
	access9.accsyst.com - - [28/Aug/1995:00:00:35 -0400] "GET /pub/robert/past99.gif HTTP/1.0" 200 4993
	access9.accsyst.com - - [28/Aug/1995:00:00:35 -0400] "GET /pub/robert/curr99.gif HTTP/1.0" 200 5836

3.3 Perancangan Metode

Proses ini menjelaskan tentang perancangan metode yang akan diajukan untuk melakukan routing antara kluster dan scalling alokasi sumber daya, berdasarkan hasil prediksi permintaan di masa mendatang dan kondisi tail latency saat ini.

3.4 Implementasi Metode

Bagian ini menjelaskan implementasi dari rancangan metode yang akan dikerjakan dari rangkaian penelitian ini. Tujuan akhir dari penelitian ini adalah membangun metode scalling dan routing traffic dinamis yang mengintegrasikan Kubernetes dan Serverless. Metode yang diusulkan terdiri dari 3

bagian:

3.4.1 Load Predictor

Load Predictor berfungsi untuk memperkirakan lalu lintas jaringan di masa mendatang berdasarkan data historis. Berikut adalah langkah-langkah implementasi:

1. Pemrosesan Data

Data historis yang digunakan dijelaskan pada bagian 3.2. Data tersebut kemudian ditransformasikan menjadi bentuk Request Per-Second (RPS) untuk mendapatkan volume traffic jaringan. GRU dipilih karena lebih efisien dalam hal penggunaan sumber daya dan waktu pelatihan (Mondal et al., 2023).

2. Struktur Model

Model menggunakan konfigurasi multi-point prediction karena terdapat cost ketika rekonfigurasi cluster sehingga konfigurasi tidak dapat dilakukan terlalu sering. Jumlah data point yang akan diprediksi adalah 30 detik ke depan.

3.4.2 Resource Allocator

Blok ini berfungsi untuk menghitung jumlah sumber daya yang harus disiapkan terhadap volume permintaan yang akan diterima selama 30 detik ke depan. Untuk memodelkan kebutuhan sumber daya, digunakan model persamaan linear:

$$R = \alpha \cdot x + \beta \quad (3.1)$$

dimana R menunjukkan jumlah sumber daya CPU yang dibutuhkan, x volume permintaan dalam satu detik, α dan β adalah koefisien pada model persamaan linear. Nilai awal α dan β didapatkan pada proses Tuning model. Saat sistem sudah berjalan, nilai koefisien akan dikoreksi berdasarkan error yang didapatkan oleh blok online controller.

3.4.2.1 Tuning Koefisien

Nilai koefisien α dan β didapatkan melalui metode OLS (Ordinary Least Squares) yang diimplementasikan menggunakan modul Scikit Linear Regression Model. Model ini menggunakan data historis dari kombinasi jumlah volume permintaan (RPS) dengan penggunaan CPU pada masing-masing layanan.

3.4.3 Online Controller

Hasil prediksi dan alokasi sumber daya oleh Load Predictor dan Resource Allocator akan selalu memiliki error yang dapat menyebabkan SLO tidak terpenuhi. Sehingga dibuat feedback controller yang berfungsi untuk menyesuaikan alokasi sumber daya dan distribusi traffic berdasarkan data monitoring.

Routing Controller Controller ini berfungsi untuk mengatur distribusi traffic antara kluster Kubernetes dengan serverless. Sistem berjalan sebagai daemon dan melakukan monitoring tiap 1 detik terhadap response tail latency. Apabila selama 5 detik berturut-turut tail latency melanggar SLO, maka sistem akan memindahkan permintaan selanjutnya hingga respons tail latency memenuhi SLO. Algoritma 3.2 menunjukkan detail mekanisme.

Cluster Controller Controller tipe ini bertanggung jawab terhadap penyesuaian alokasi sumber daya pada kluster Kubernetes terhadap penggunaan sekarang. Sistem akan melakukan monitoring terhadap tail latency, kemudian setiap 2 detik dilakukan perhitungan ulang kebutuhan sumber daya berdasarkan nilai SLO violation. Detail algoritma dapat dilihat pada bagian 2.5.2.

3.5 Rencana Evaluasi

Untuk mengukur kinerja metode ini, dilakukan 2 tahap evaluasi: evaluasi untuk komponen traffic predictor dan evaluasi untuk komponen routing dan

```

1: Variables:
2:   reroute_traffic  $\leftarrow$  False
3:   violation_timer  $\leftarrow$  0
4:   compliance_timer  $\leftarrow$  0
5: repeat
6:   tail_latency  $\leftarrow$  GetTailLatency()
7:   CPU_usage  $\leftarrow$  GetCPUUsage()
8:   if tail_latency > SLO then
9:     violation_timer  $\leftarrow$  violation_timer + 1
10:    compliance_timer  $\leftarrow$  0
11:   else
12:     compliance_timer  $\leftarrow$  compliance_timer + 1
13:     violation_timer  $\leftarrow$  0
14:   end if
15:   if violation_timer  $\geq$  5 and not reroute_traffic then
16:     RerouteToServerless()
17:     reroute_traffic  $\leftarrow$  True
18:   else if violation_timer  $\geq$  5 and not reroute_traffic then
19:     RerouteToKubernetes()
20:     reroute_traffic  $\leftarrow$  False
21:   end if
22:   wait(1) ▷ Delay for 1 second
23: until forever

```

Gambar 3.2 Routing Controller

scaling.

3.5.1 Evaluasi Traffic Predictor

Evaluasi traffic predictor bertujuan untuk mengukur kinerja model regresi berbasis GRU pada dataset jumlah permintaan per detik (RPS). Evaluasi kinerja model dilakukan menggunakan metode perhitungan Root Mean Square Error (RMSE). Nilai RMSE didapat dari rata-rata perbedaan hasil prediksi dengan nilai aktual. Nilai yang rendah menunjukkan hasil prediksi yang lebih akurat.

3.5.2 Evaluasi Router dan Scalling

Bagian ini bertujuan untuk mengukur performa metode pengambilan keputusan dalam melakukan routing ke kluster lain atau perubahan pada alokasi sumber daya. Pengukuran performa dilakukan dengan menganalisis beberapa komponen metrik:

1. Request Running Time: Waktu yang dibutuhkan untuk melayani permintaan, dimulai dari saat permintaan diterima hingga selesai dilayani.
2. Tail Latency: Mengukur waktu respons terburuk yang biasanya berada pada persentil tinggi (misalnya, 99th percentile).
3. Alokasi CPU dan RAM untuk Kluster: Mengukur alokasi sumber daya CPU dan RAM yang dialokasikan untuk setiap kluster.
4. Penggunaan CPU pada Kluster: Mengukur persentase penggunaan CPU pada setiap kluster selama periode pengujian.
5. Distribusi Permintaan: Mencakup kemampuan sistem dalam mendistribusikan permintaan antara kluster serverless dan kluster Kubernetes.
6. Jumlah Permintaan: Total jumlah permintaan yang diterima dan diproses oleh sistem dalam periode tertentu.
7. Jenis metode penempatan (Variabel Kontrol): Metode deployment yang digunakan — full serverless, full Kubernetes kluster, dan hybrid.
8. Jenis metode scalling (Variabel Kontrol): Mencakup cluster autoscaler,

metode ELAX, dan metode yang diusulkan dari modifikasi ELAX.

3.6 Penyusunan Laporan

Tahapan ini merupakan tahapan setelah penelitian selesai. Pada bagian ini dilakukan penulisan hasil sebagai laporan dari seluruh tahapan yang sudah dilakukan dalam penelitian.

3.7 Rencana dan Jadwal Kegiatan Penelitian

Pelaksanaan penelitian tesis ini akan dilakukan berdasarkan jadwal pekerjaan yang dirancang sebelumnya. Durasi pelaksanaan kegiatan penelitian ini direncanakan akan dikerjakan selama 6 bulan dengan detail pada Tabel 3.2.

Tabel 3.2 Rencana Kegiatan Penelitian Thesis

No	Nama Kegiatan	Bulan ke-					
		1	2	3	4	5	6
1	Studi literatur						
2	Pengumpulan Data						
3	Perancangan Sistem						
4	Implementasi dan Pengu- jian						
5	Penyusunan jurnal ilmiah						
6	Penyusunan Laporan						

DAFTAR PUSTAKA

- Aslanpour, M. M., Gill, S. S. and Toosi, A. N. (2020), ‘Performance evaluation metrics for cloud, fog and edge computing: A review, taxonomy, benchmarks and standards for future research’, *Internet of Things* 12.
- He, Z. (2020), Novel container cloud elastic scaling strategy based on kubernetes, in ‘2020 IEEE 5th Information Technology and Mechatronics Engineering Conference (ITOEC)’, pp. 1400–1404.
- Kavis, M. (2014), ‘Cloud services and division of responsibilities’.
- Malathi, M. (2011), Cloud computing concepts, in ‘2011 3rd International Conference on Electronics Computer Technology’, pp. 236–239.
- Mampage, A., Karunasekera, S. and Buyya, R. (2022), ‘A holistic view on resource management in serverless computing environments: Taxonomy and future directions’, *ACM Computing Surveys* 54(11s).
- Mell, P. M. and Grance, T. (2011), The nist definition of cloud computing, Technical report, National Institute of Standards and Technology.
- Mondal, S. K., Wu, X., Kabir, H. D., Dai, H.-N., Ni, K., Yuan, H. and Wang, T. (2023), ‘Toward optimal load prediction and customizable autoscaling scheme for kubernetes’, *Mathematics* 11(12).
- Pahl, C., Brogi, A., Soldani, J. and Jamshidi, P. (2019), ‘Cloud container technologies: A state-of-the-art review’, *IEEE Transactions on Cloud Computing* 7(3), 677–692.
- Sadaqat, M., Colomo-Palacios, R. and Knudsen, L. E. (2018), ‘Serverless com-

puting: a multivocal literature review’.

URL: <https://api.semanticscholar.org/CorpusID:86762715>

Savage, N. (2018), ‘Going serverless’, *Communications of the ACM* 61, 15–16.

Senjab, K., Abbas, S., Ahmed, N. and Khan, A. u. R. (2023), ‘A survey of kubernetes scheduling algorithms’, *Journal of Cloud Computing* 12(1).

Sewak, M. and Singh, S. (2018), Winning in the era of serverless computing and function as a service, in ‘2018 3rd International Conference for Convergence in Technology (I2CT)’, pp. 1–5.

Smit, M. (2013), Supporting software evolution to the multi-cloud with a cross-cloud platform, in ‘2013 IEEE 7th International Symposium on the Maintenance and Evolution of Service-Oriented and Cloud-Based Systems’, pp. 85–85.

Wurster, M., Breitenbücher, U., Kepes, K., Leymann, F. and Yussupov, V. (2018), Modeling and automated deployment of serverless applications using toasca, in ‘2018 IEEE 11th Conference on Service-Oriented Computing and Applications (SOCA)’, pp. 73–80.

Yadav, A. K., Garg, M. L. and Ritika (2019), Docker containers versus virtual machine-based virtualization, in ‘Advances in Intelligent Systems and Computing’, Springer Singapore, pp. 141–150.

Yang, Y., Zhao, L., Li, Z., Nie, L., Chen, P. and Li, K. (2019), ElaX: Provisioning resource elastically for containerized online cloud services, in ‘Proceedings - 21st IEEE International Conference on High Performance Computing and Communications, 17th IEEE International Conference on Smart City and 5th IEEE International Conference on Data Science and Systems, HPCC/SmartCity/DSS 2019’, pp. 1987–1994.

Yu, T., Liu, Q., Du, D., Xia, Y., Zang, B., Lu, Z., Yang, P., Qin, C. and Chen, H. (2020), Characterizing serverless platforms with ServerlessBench,

in ‘SoCC 2020 - Proceedings of the 2020 ACM Symposium on Cloud Computing’, pp. 30–44.

Yuan, H. and Liao, S. (2024), ‘A time series-based approach to elastic kubernetes scaling’, *Electronics* 13(2), 285.

Zhang, X., Li, L., Wang, Y., Chen, E. and Shou, L. (2021), ‘Zeus: Improving resource efficiency via workload colocation for massive Kubernetes clusters’, *IEEE Access* 9, 105192–105204.